Polyglot

A compiler for mixed syntax

Compiler Design Project

Source code, typed IR, optimization and a custom virtual machine

The basic idea

Different syntax forms share one language model

Many familiar spellings

```
print(value)
System.out.println(value);
std::cout << value << std::endl;
console.log(value);
PRINT value
```

One compiler

The parser normalizes syntax. Shared type rules, IR and bytecode determine the result.

Selected syntax from Python, C, C++, Java, JavaScript and BASIC.

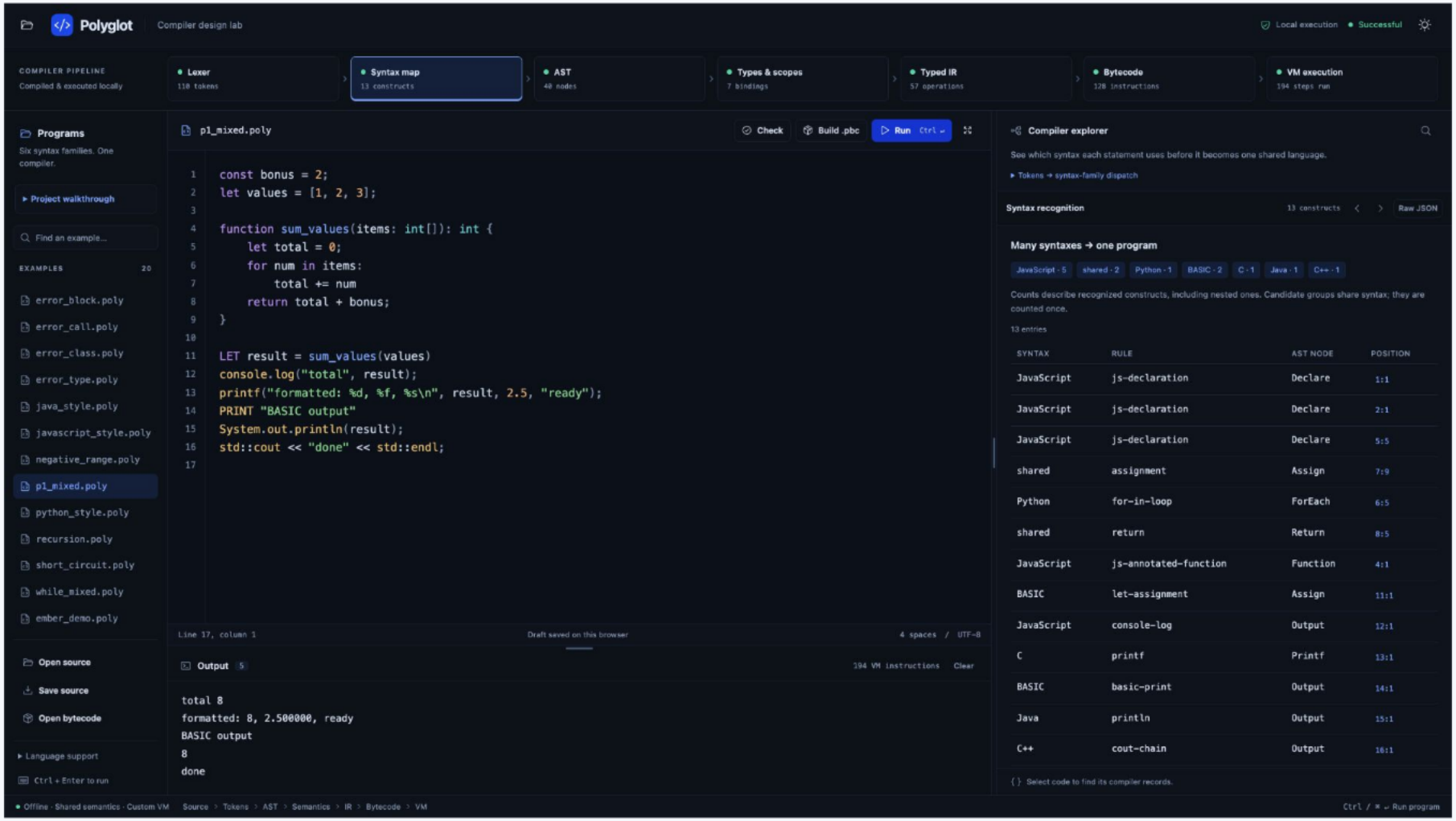
Supported language subset

A deliberately bounded grammar with real compilation and execution

Area	Implemented support
Values	int, float, bool, string and one-dimensional arrays
Functions	Typed parameters and returns, forward calls and recursion
Control flow	if / else, while, range, C-style for, break and continue
Declarations	Inferred or explicit types, let / const and BASIC LET
Blocks and output	Mixed indentation / braces and syntax-specific output forms

Every syntax uses the same binding, type and runtime rules.

The working compiler workspace



Compilation pipeline

Each phase consumes the representation produced by the previous phase

Phase	Output
Lexical analysis	Tokens with text and source spans
Parsing and normalization	Shared abstract syntax tree
Semantic analysis	Resolved symbols and checked types
IR generation	Typed three-address instructions
Optimization	Simplified IR with a pass report
Code generation	Validated .pbc stack bytecode
Execution	Program output or a mapped runtime diagnostic

Mixed syntax in one program

A function written with one syntax can serve another syntax

```
int add(int a, int b) {
    return a + b;
}

def twice(value: int) -> int:
    return add(value, value)

int result = twice(6);
std::cout << result << std::endl;
print("done")
```

Output

```
12
done
```

Both calls resolve through the same symbol table and VM.

Lexical analysis

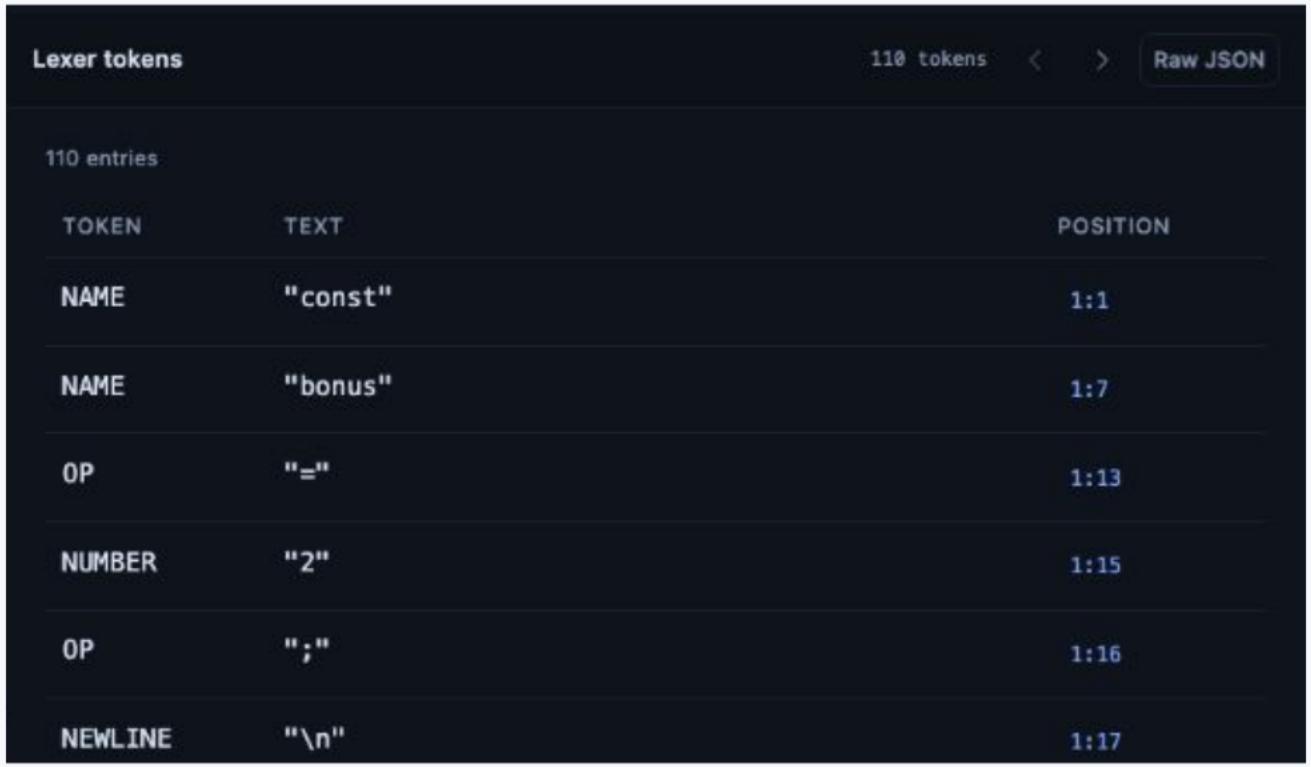
Characters become tokens that retain their source positions

Token categories

Names, literals, operators and layout markers.

Source spans

Locations support diagnostics and editor links.



Parsing and operator precedence

Recursive descent handles statements, while Pratt parsing handles expressions

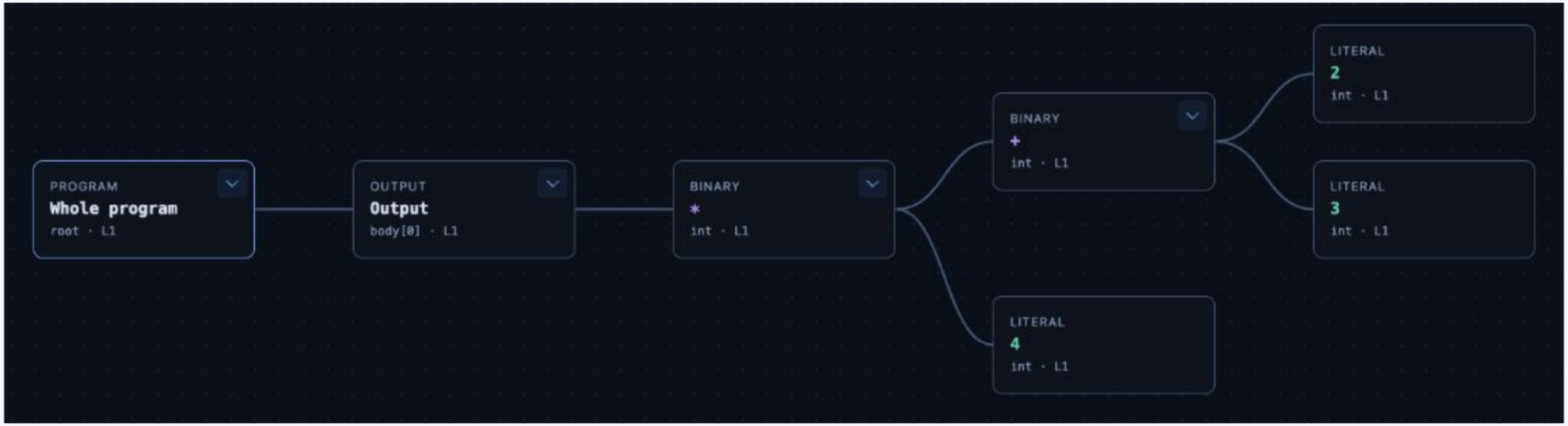
Decision	Implementation
Statement family	Leading tokens and bounded lookahead
Expression structure	Binding powers encode operator precedence
Mixed blocks	Context-aware suite reader handles indentation and braces
Normalization	Family-specific spelling creates shared AST nodes

```
2 + 3 * 4
```

The multiplication groups before the addition.

The AST is a real tree

print((2 + 3) * 4)



Semantic structure

Operations and operands replace punctuation.

Source connection

Types and source links stay attached to each node.

Semantic analysis

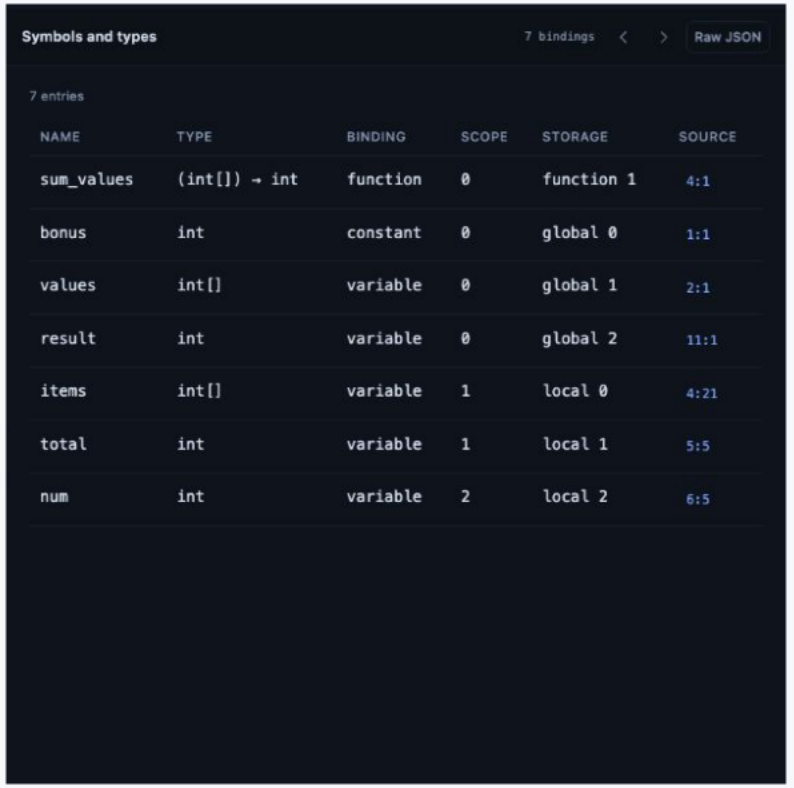
A well-formed tree must also obey binding and type rules

Name resolution

Nested scopes distinguish globals, parameters and local variables.

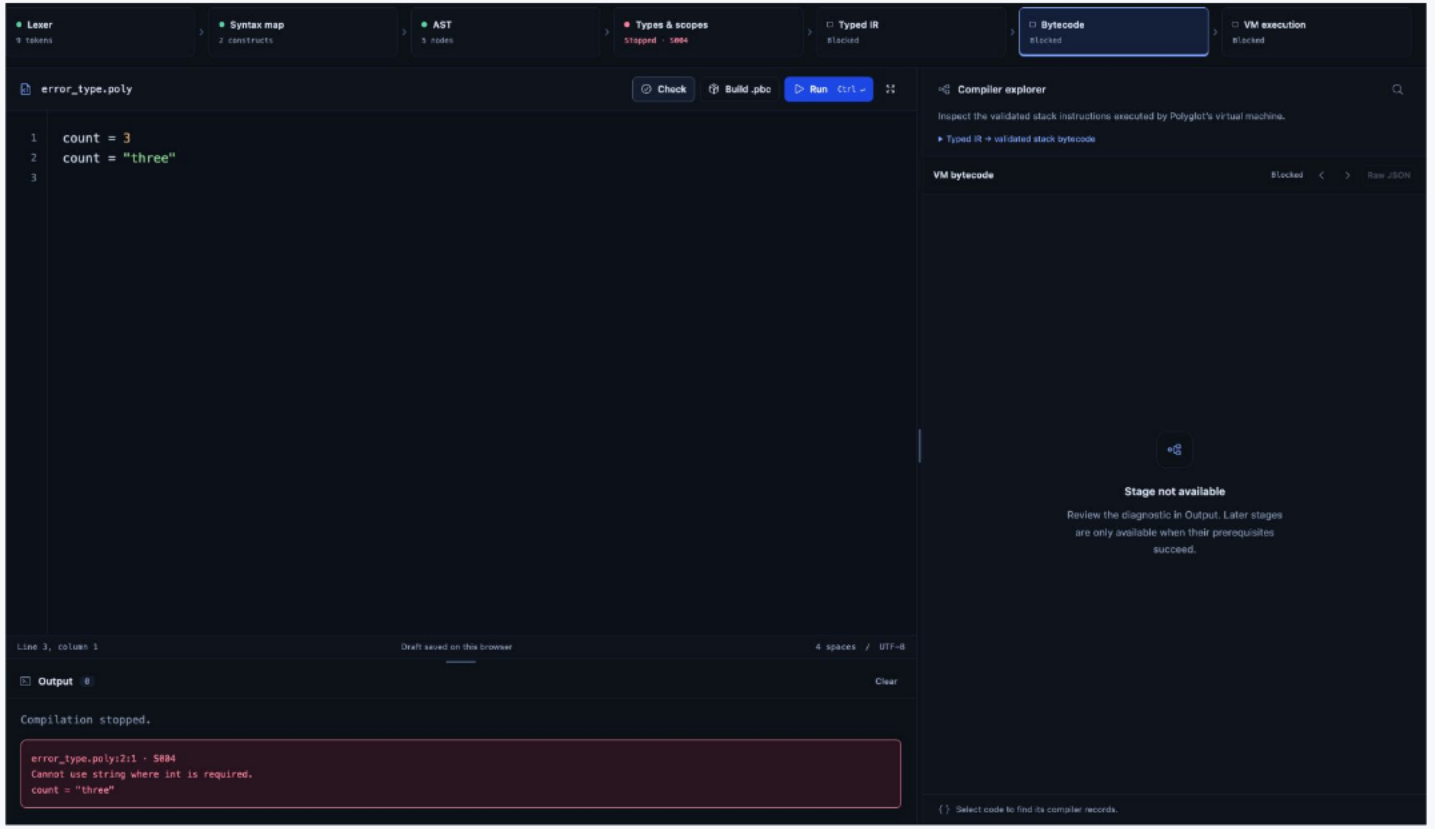
Type checking

Assignments, calls, returns and array operations must use compatible types.



Semantic errors stop code generation

The source parses successfully, but assigning a string to an int fails



S004 at line 2: Cannot use string where int is required.

Typed three-address IR

The compiler expresses computation using explicit temporaries and operations

```
t0 = 2              : int
t1 = 3              : int
t2 = t0 + t1        : int
t3 = 4              : int
t4 = t2 * t3        : int
output(t4)
return
```

Readable notation for the expression.
The inspector shows original IR.



Typed intermediate representation — 57 operations — Raw JSON

Readable three-address notation · t = temporary · global[n] / local[n] = storage slot. Original operands are preserved in Raw JSON.

▼ <main>() → void

31 entries

#	THREE-ADDRESS CODE	OPERATION	SOURCE
0	t0 = 2 : int	CONST	1:15
1	global[0] = t0	STORE	1:1
2	t1 = 1 : int	CONST	2:15
3	t2 = 2 : int	CONST	2:18
4	t3 = 3 : int	CONST	2:21
5	t4 = [t1, t2, t3] : int[]	ARRAY_BUILD	2:14
6	global[1] = t4	STORE	2:1
7	t5 = global[1] : int[]	LOAD	11:25
8	t6 = call sum_values(t5) : int	CALL	11:14
9	global[2] = t6	STORE	11:1
10	t7 = "total" : string	CONST	12:13
11	t8 = global[2] : int	LOAD	12:22

Control flow and data-flow analysis

These analyses support actual optimization decisions

Analysis	How the compiler uses it
Basic blocks	Split at entry, labels and instructions after terminators
Control-flow graph	Resolve branch, jump and fall-through successors
Reachability	Remove blocks with no path from function entry
Liveness and next use	Remove only safe unused constant definitions

```
OUT[B] = union of IN[S] for each successor S
IN[B]  = USE[B] union (OUT[B] - DEF[B])
```

Implemented optimization passes

Optimization runs on typed IR before bytecode generation

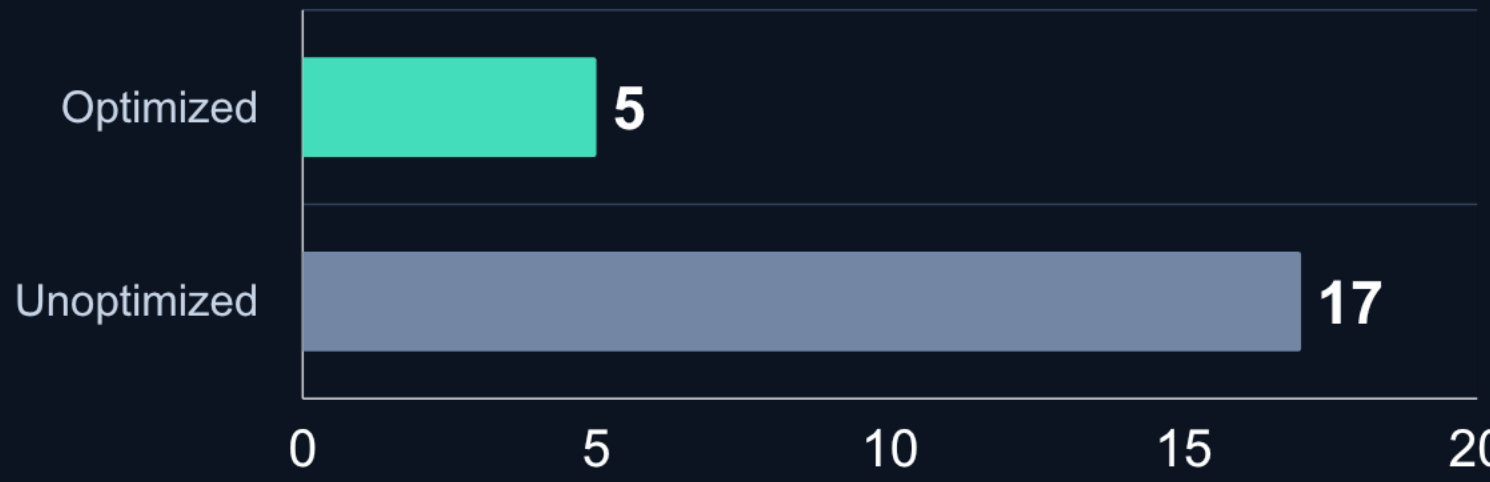
Pass	Actual transformation
Local constant folding	Replace a known successful operation with a typed constant
Constant branch simplification	Replace a proven Boolean branch with a direct jump
Unreachable code removal	Rebuild the graph and discard unreachable blocks
Safe dead-constant removal	Use liveness and next use to remove unused CONST definitions

Reachable calls, output, mutable loads and runtime checks remain.

17 instructions become 5

Same source. Same output. Fewer executed VM instructions.

```
print((2 + 3) * 4)
```



20

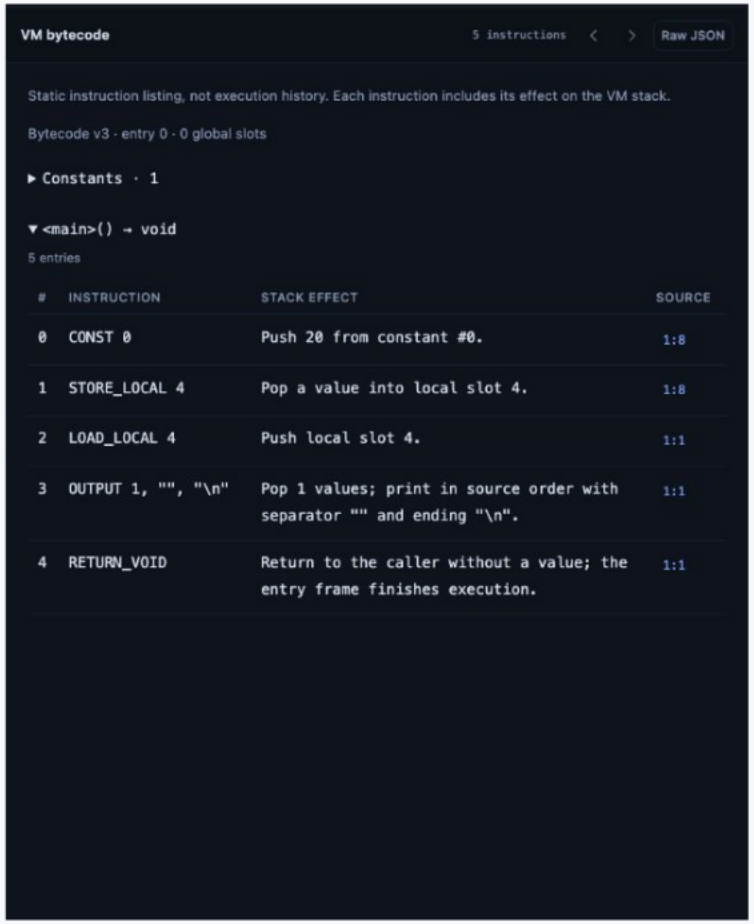
output in both runs

70.6% fewer instructions in this example. This is an instruction count, not a timing benchmark.

Generated stack bytecode

The executable representation contains one computed constant

```
CONST 0
STORE_LOCAL 4
LOAD_LOCAL 4
OUTPUT 1, "", "\n"
RETURN_VOID
```



Constant pool entry 0 contains 20.

Virtual machine and activation records

Each function call receives its own execution frame

Frame field	Purpose
function	Selected bytecode and function metadata
locals	Parameters, local variables and IR temporary slots
ip	Next instruction to execute
stack	Operand values for this call
callsite	Source location used in runtime diagnostics

```
factorial(6)

6 * factorial(5)
5 * factorial(4)
...
return 720
```

Fresh frames keep recursive calls separate.

Arrays across syntax families

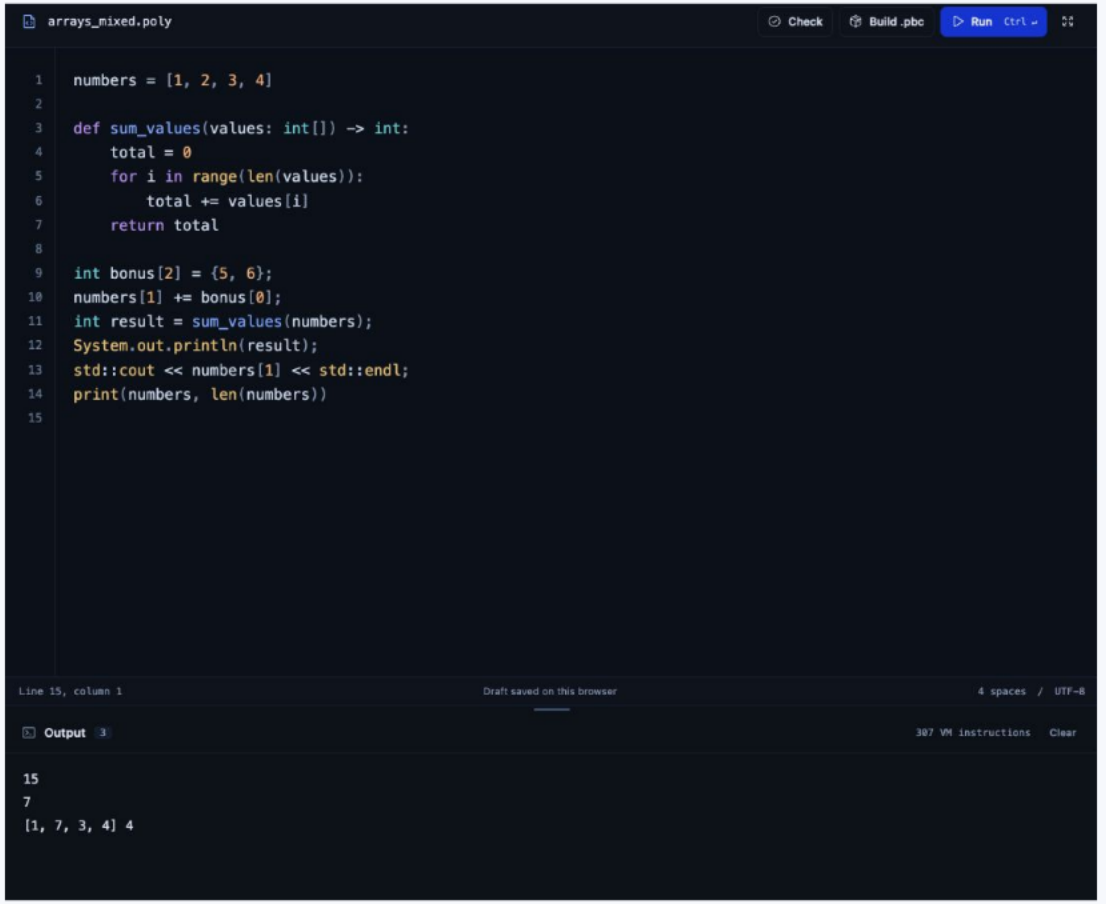
Shared references and checked indexing use dedicated IR and VM operations

Fixed element type

An int[] stays an int[] across declarations, assignment and function calls.

Runtime checks

The VM checks index types, bounds, element types and allocation limits.



A reusable .pbc artifact

Compilation and execution can happen separately

```
python run_polyglot.py build examples/mixed_demo.poly -o build/demo.pbc

python run_polyglot.py exec build/demo.pbc
```

Artifact contents

Constants, function code, storage metadata and source mappings.

Independent execution

The VM validates and executes the artifact without reading the source file.

Local application architecture

The CLI and browser share the compiler library

Component	Responsibility
Browser UI	Plain HTML, CSS and JavaScript with bundled offline assets
Loopback server	Routes requests and checks Host / Origin
Spawned worker	Compiles or runs with a parent wall-clock deadline
Compiler library	One implementation for CLI and web operations
Bounded VM	Validates bytecode and enforces execution limits

Runtime requirement: Python 3.11 or newer. No third-party runtime dependencies.

Verification

Local verification snapshot from 26 October 2026

296

Python tests passed

54

Frontend tests passed

Compiler behavior, HTTP workers, artifact execution and editor interactions

Optimization tests compare outputs, errors, evaluation order and all valid bundled examples.

Compiler-design concepts used

The concepts appear in the compiler implementation itself

Concept	Concrete implementation
Lexical and syntax analysis	Scanner, recursive descent, Pratt precedence and shared AST
Semantic analysis	Scoped symbols, fixed types, call / return validation
Intermediate code	Typed three-address operations and symbolic control flow
Optimization and data flow	Basic blocks, CFG, liveness, next use and constant folding
Code generation	Stack instructions and deferred jump-target patching
Runtime organization	Global slots, local slots, call frames and operand stacks

Demonstration sequence

A short walkthrough with observable results

Demo	What it demonstrates	Expected result
p1_mixed.poly	Six syntax families in one source	Five output lines
mixed_nesting.poly	Braces and indentation in nested blocks	22
error_type.poly	A semantic failure blocks later phases	S004 at line 2
print((2 + 3) * 4)	Original and optimized compilation	20 with 17 vs 5 steps

Finish by building a .pbc file and executing it independently.

Current limits and next steps

The implementation has a clear, testable boundary

Language boundary

Selected constructs only. Classes, imports and general object operations remain unsupported.

Compiler boundary

One handwritten parser strategy and a custom stack VM. No LLVM or LR parser-table backend.

Next compiler work

Multiple-error recovery, broader data-flow analysis and further safe optimizations.

Next engineering work

Larger benchmark programs and more diagnostics for grammar extensions.

What Polyglot demonstrates

A complete compiler pipeline for a defined mixed-syntax language

One shared semantic model

Actual IR optimization

Independent bytecode execution

Working evidence is available in the editor, compiler stages, tests and artifacts.

Appendix: implementation map

Files to open during a code walkthrough

Area	Source location
Compiler orchestration	src/polyglot/compiler.py
Scanner and parsing	lexer.py and parser/
Semantic rules and lowering	semantics.py and ir.py
Data flow and optimization	analysis.py and optimizer.py
Target and execution	codegen.py, artifact.py and vm.py
Web application	web/server.py, web/worker.py and web/static/

Appendix: viva questions

Short answers grounded in the implementation

Compiler or interpreter?

The front end compiles to .pbc. A custom VM interprets that bytecode.

Why a shared AST?

It removes syntax differences before common semantic checking and code generation.

Why optimize the IR?

Typed operations expose constants and control flow before stack lowering.

How is correctness checked?

Differential tests compare output and errors with optimization enabled and disabled.

Appendix: sources and attribution

Project foundation

hst257/PolyGlot on GitHub
MIT License, © 2026 Harshit and Polyglot contributors

Implementation evidence

Current local source, architecture notes and verification report
Live application screenshots captured on 26 October 2026

The local workspace includes editor improvements and a conservative IR optimizer. Local changes remain unpushed.